Reorder the Array, so that
Prime Numbers are placed at beginning.

```
void input(int *, int);
 main() {   int  arr[10];
             input(arr, 10);✓
             print(arr, 10);
             reorder(-);  sort(arr, j);
             print(arr, 10);
        }.
```

2-40

```
void input(int *ptr, int n)
   {  int i; srand(getpid());
 for(i=0; i<n; i++){ ptr[i]= rand() %39 +2;
                      if(! isunique(ptr, i)) {
                                               --i;
   }                                         }
```

```
int isunique(int *p, int i) { int j; //verify p[i] is unique or not,
                   //by comparing p[i] with p[0], p[1], ....p[i-1]
                // if repeatation found, then return 0, else
                                                          retun 1
   for(j=0; j<i; j++) if (p[j] == p[i])
                                         return 0;

            return 1; }
```

MUST TRY before 22nd

→ sort the prime no. Portion of array in the - above program.

→ Find the 2nd Highest element in an Array of 10 Random Numbers.

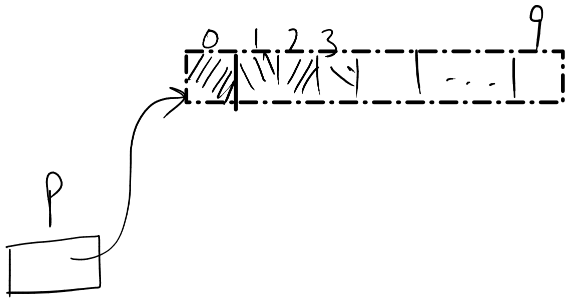
- → method 1 : Random, may not be Unique, hence 2nd Highest may have repeatations
- → method 2: Random, Unique, Sample of data

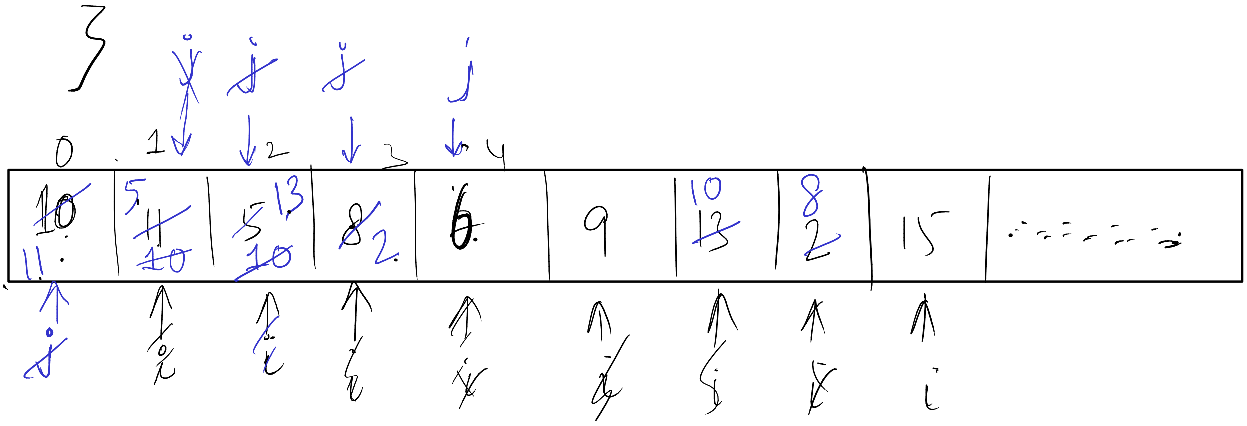
logic 1: Find the highest : loop 1
find the 2nd Highest : loop 2

logic 2: Complete Using Single loop ✓

```
void print(int *p, int n)
{ // Display all integers present in the array (referred by p)
    int i;
for(i=0; i<n; i++)
  pf("%d, ", p[i]);

}
```





```
void reorder(int *p, int n)
{
        int i, j;
        j=0;
for ( i=0; i<n; i++) {
                          if( checkPrime(p[i]))
                          { int temp=p[i];
                            p[i]=p[j]; p[j]=temp;
                          ++j;
                          }

}
```

Sorting (Arrangement of data in an order)

- Ascending
- descending

int a[1] = {20};

int b[2] = {40, 30}; Sorted

int c[3] = {40, 20, 30};

0	1	2
~~40~~	~~20~~	30
20	~~40~~ 30	~~30~~ 40

0 - 1	0 - 1
1 - 2	

Sorting Algorithms

- heap
- merge
- quick
- bubble
- selection
- insertion

Bubble Sort

→ Compare Adjacent elements

~~50~~, ~~40~~, ~~30~~

40, ~~50~~, 30

~~40~~, ~~30~~, 50

$\boxed{30}$, $\boxed{40}$, $\boxed{50}$

0	1 ✓	2 ✓	$\boxed{3}$ ✓
~~50~~ 10	~~10~~ ~~50~~	~~40~~ 20	~~20~~ $\boxed{50}$
	40	~~50~~	

10, ~~40~~, ~~20~~, $\boxed{50}$

20 $\boxed{40}$

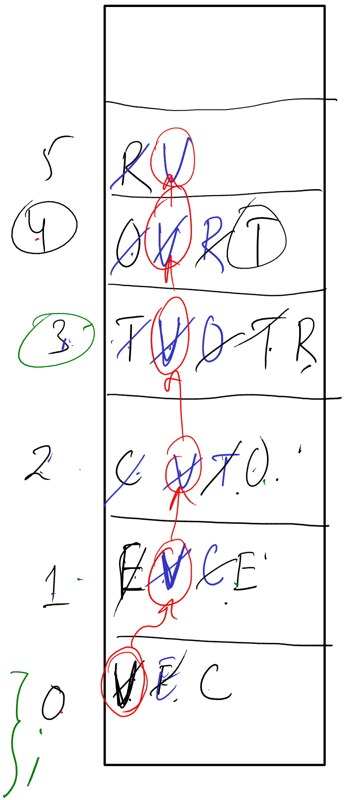
10, $\boxed{20}$, 40, 50

3	2	1
0 - 1	0 - 1	0 - 1
1 - 2	1 - 2	
2 - 3		

0	1	2	3	4	5
'V'	'E'	'C'	'T'	'O'	'R'

n => no-of-elements => 6

5	4	3	2	1
0-1	0-1	0-1	0-1	0-1
1-2	1-2	1-2	1-2	
2-3	2-3	2-3		
3-4	3-4			
4-5				



```
char arr[] = {'V','E','C','T','O','R'};

n = sizeof(arr)/ sizeof(arr[0]);
```

n=6

n=6

i	5	4	3	2	1
j	0-1	0-1	0-1	0-1	0-1
	1-2	1-2	1-2	1-2	
	2-3	2-3	2-3		
	3-4	3-4			
	4-5				

```
~~int~~ char temp;
for(i=n-1; i>=1; i--)
{
   flag=1;
for(j=0; j<i; j++)
{
    if(arr[j] > arr[j+1])
    {
      temp= arr[j];
      arr[j]=arr[j+1];
      arr[j+1]= temp; flag=0;
    }
}
if(flag==1) break;
}
```

'A'	'B'	'C'	'D'	'E'	'f'